



CSC220 · WEB DEVELOPMENT II · WEEK 3 · SESSION 1

Git & GitHub

Version Control Fundamentals • Repositories & Commits

git

Stamford International University — Information Technology / Computer Science

Lecturers: Thinzar Aung Win · Zakariyya Bature

Today's Agenda

This session introduces Git, a version control system used by developers to keep track of changes in their projects.

Students will learn:

- Why developers use version control
- What Git repositories and commits are
- How Git tracks changes
- How to create their first Git repository
- How to view previous changes
- How to ignore unnecessary files
- How to safely undo mistakes

This session focuses mainly on Git on your local computer.

All Git commands can be entered using the VS Code integrated terminal.

NEXT SESSION GitHub and team collaboration will be covered in the next session.



Why Do We Need Version Control?

Imagine you are working on a project and you save files like:

```
project.zip  
project_final.zip  
project_final_v2.zip  
project_final_v2_REAL.zip
```

This becomes confusing very quickly.

You may not know:

- Which file is the newest
- What changed between the files
- Which version works correctly
- Which version your teammate has

Git solves these problems.

Problem 1: Too Many Backup Files

Without Git, students may create many copies of the same project.

For example:

```
project/  
project_backup/  
project_final/  
project_final2/  
project_really_final/
```

With Git, you normally keep one project folder.

Git remembers the previous versions for you.



Why Do We Need Version Control?

Problem 2: "It Worked Yesterday"

Sometimes you change your code and something stops working.

Without Git, it may be difficult to remember exactly what you changed.

With Git, you can check:

- What changed
- When it changed
- Who changed it

You can also return to an earlier working version.

Problem 3: "Who Has the Latest Copy?"

This becomes especially difficult in group projects.

Git helps everyone work with the same project history.

Later, GitHub will allow the repository to be shared online.



Why Do We Need Version Control?

Git vs GitHub

An important difference:

Git

Git is the version control system.
It runs on your computer.

GitHub

GitHub is an online service that hosts Git repositories.

A simple way to remember it:

```
Git = manages versions  
GitHub = stores and shares Git repositories online
```

They are related, but they are not the same thing.



How Git Thinks

This is one of the most important concepts in Git.

Git works with three main areas:





How Git Thinks

1. Working Directory

This is where you are currently editing your files.

For example:

```
index.html
style.css
app.js
```

If you change `app.js`, the change starts in the working directory.

Think of it as:

My current project files.

2. Staging Area

The staging area contains changes that you want to include in your next commit.

You move changes to the staging area using:

```
● ● ●
$ git add
```

For example:

```
● ● ●
$ git add app.js
```

This tells Git:

I want the changes in `app.js` to be included in my next saved version.



How Git Thinks

3. Repository / History

When you commit your staged changes, Git permanently records them in the project history. You use:

```
● ● ●  
$ git commit
```

What Is a Commit?

A commit is a saved snapshot of your project at a particular moment.

For example:

```
Commit 1 → Create webpage  
Commit 2 → Add navigation bar  
Commit 3 → Add API request  
Commit 4 → Add error handling
```

Each commit includes information such as:

- A unique ID
- Author
- Date
- Commit message
- Changes to the project

Simple Analogy

Think of the staging area as packing a box.

You choose which things you want to put inside:

```
● ● ●  
$ git add
```

Then committing is like sealing the box and putting a label on it:

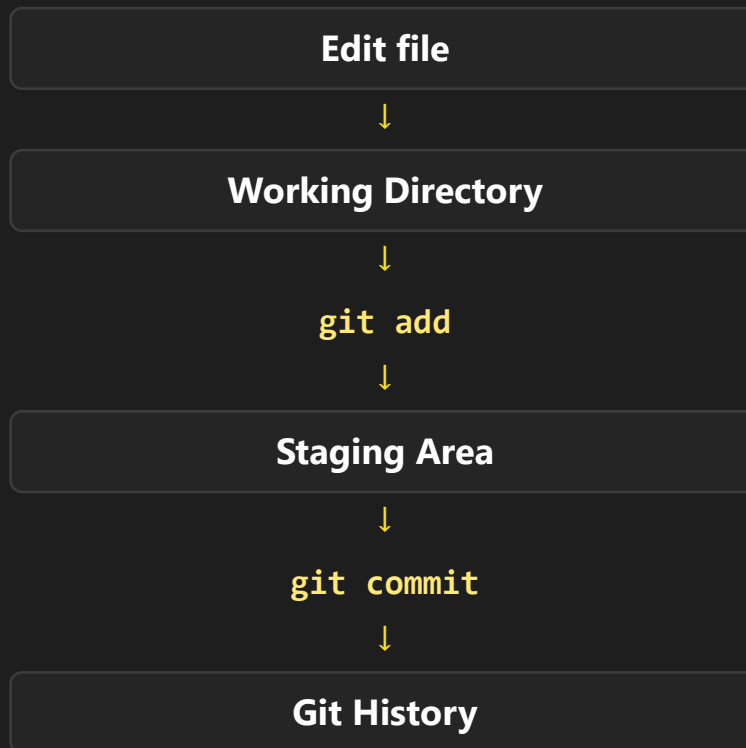
```
● ● ●  
$ git commit
```

Once the box is saved, it becomes part of your project history.



How Git Thinks

The flow becomes:





How Git Thinks

What Is the .git Folder?

When Git manages a project, it creates a hidden folder called:

```
.git
```

This folder contains the Git history and repository information.

You normally do not edit this folder manually.

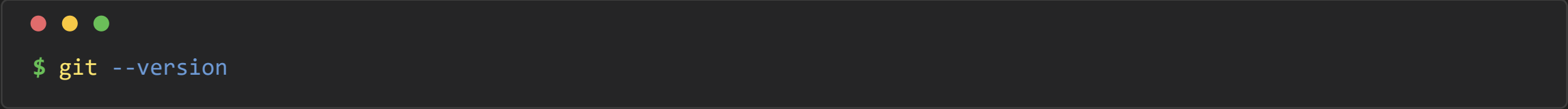
If you delete .git, the project files remain, but the Git history is lost.



Installing and Configuring Git

First, check whether Git is installed.

Open the VS Code terminal and run:



```
$ git --version
```

You may see something like:

```
git version 2.45.0
```

That means Git is installed correctly.



Installing and Configuring Git

Tell Git Who You Are

Git records the author of every commit.

Set your name:



```
$ git config --global user.name "Wendy"
```

What Does --global Mean?

--global means:

Use this setting for every Git project on this computer.

You normally need to configure this only once.

Set your email:



```
$ git config --global user.email "wendy@stamford.edu"
```

Students should use their real name and university email.



Installing and Configuring Git

Set the Default Branch



```
$ git config --global init.defaultBranch main
```

This tells Git to call the default branch:

```
main
```

Check Your Settings

Use:



```
$ git config --list
```

This displays your current Git configuration.

You should see your username and email among the settings.



Your First Git Repository

Suppose your project folder is:

```
my-project
```

Move into the folder:

```
● ● ●  
$ cd my-project
```

Step 1: Initialize Git

Run:

```
● ● ●  
$ git init
```

This tells Git:

```
Start tracking this project.
```

Git creates the hidden .git folder.



Your First Git Repository

You only need to run git init once for each project.

Step 2: Check the Status

Run:



```
$ git status
```

Git may show:

```
untracked:
index.html
app.js
```

untracked means:

Git can see these files, but Git is not tracking them yet.



Your First Git Repository

Why git status Is Important

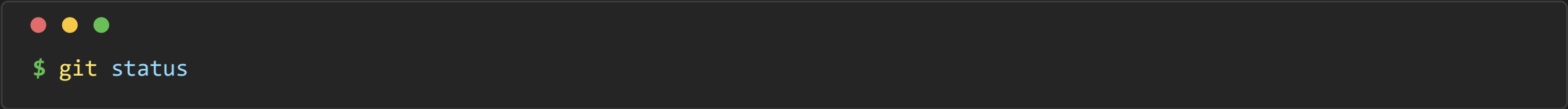
git status tells you what is happening in your repository.

It can tell you:

- Which files are new
- Which files were changed
- Which files are staged
- Which files are not staged

It is one of the safest and most useful Git commands.

A good habit is to frequently run:



```
$ git status
```




Your First Git Repository

Step 3: Stage Files

To stage one file:



```
$ git add app.js
```

To stage all changed files:



```
$ git add .
```

The . means:

Add all changes in the current project.

Step 4: Commit

After staging your changes, save them to Git history:



```
$ git commit -m "Add fetch-based user list"
```



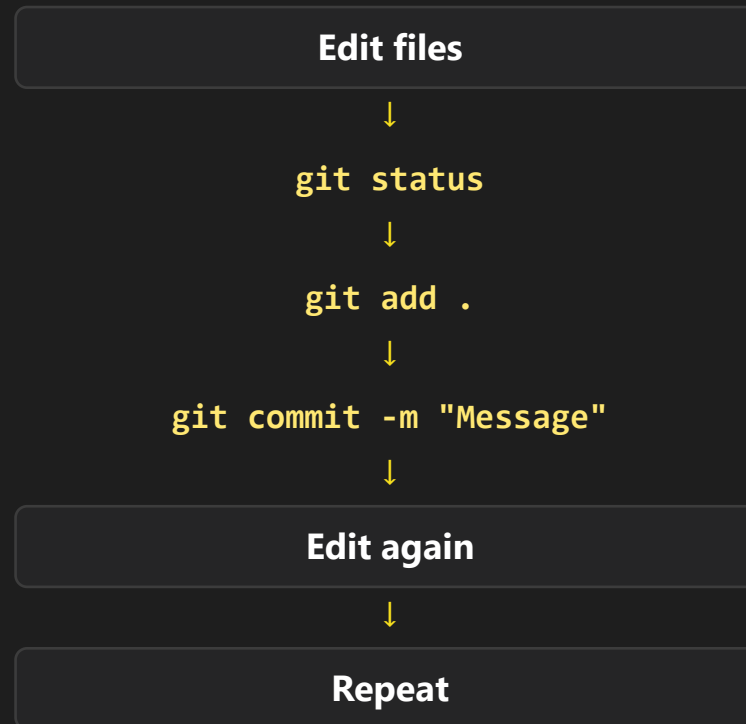
Your First Git Repository

The -m means you are providing a commit message.

The message explains what you changed.

The Daily Git Cycle

This is the main Git workflow students should remember:





Your First Git Repository

You run:

```
● ● ●  
$ git init
```

only once.

But you repeat:

```
● ● ●  
$ git status  
$ git add  
$ git commit
```

many times while developing your project.

Good Commit Messages

Commit messages should clearly explain the change.



Your First Git Repository

Good

- Add login form
- Add loading message
- Fix navigation menu
- Handle fetch errors
- Create user profile page

Poor

- stuff
- update
- changes
- final
- final2
- done

A good message should tell another developer what changed.

Commit Small and Often

Do not wait until the whole project is finished before committing. Instead, create commits for logical changes.

For example:

```
Initial commit
Add navigation bar
Add API request
Display user data
Add loading state
Handle API errors
```

This creates a useful project history.



Viewing Git History

Git can show all previous commits.

Use:



```
$ git log --oneline
```

Example:

```
3f9e2d1 Add error handling to fetch  
1a2b3c4 Add fetch-based user list  
9c8d7e6 Initial commit
```

The newest commit appears first.

Understanding a Commit ID

This part:

```
3f9e2d1
```

is the short version of the commit's unique ID.



Viewing Git History

Git uses this ID to identify a specific commit.

You can use the first few characters because they are normally enough to uniquely identify the commit.

git diff

Use:

```
● ● ●  
$ git diff
```

to see changes you made but have not staged yet.

Example:

```
- list.innerHTML = "";  
+ list.innerHTML = "<li>Loading...</li>";
```

The - line shows the old version.

The + line shows the new version.

So you changed:

```
list.innerHTML = "";
```

to:

```
list.innerHTML = "<li>Loading...</li>";
```



Viewing Git History

git diff --staged

Use:



```
$ git diff --staged
```

to see changes that are already in the staging area.

This is useful before committing because you can check:

What exactly am I about to commit?

git show

Use:



```
$ git show 1a2b3c4
```

to inspect one previous commit.

It shows what changed in that commit.



.gitignore

Sometimes there are files that Git should not track.

We can tell Git to ignore them using a file called:

```
.gitignore
```

The file should normally be placed in the project's root folder.

Example .gitignore

```
node_modules/  
.env  
.DS_Store  
*.log  
dist/
```

node_modules/

```
node_modules/
```

This folder can contain thousands of dependency files.

You normally do not commit it because it can be recreated using npm.



.gitignore

.env

```
.env
```

A .env file may contain sensitive information such as:

- API keys
- Passwords
- Database credentials

Therefore, it should not be committed.

.DS_Store

```
.DS_Store
```

This is a file created by macOS.

It is not part of your application, so Git does not need to track it.

*.log

```
*.log
```

This means:

Ignore all files ending with .log.

For example:

```
server.log  
error.log  
debug.log
```

dist/

```
dist/
```

This folder often contains generated application files.

Because these files can normally be generated again, they often do not need to be tracked.



.gitignore

What Should Git Track?

Git should mainly track your source code.

For example:

```
index.html  
style.css  
app.js  
package.json  
README.md
```

Avoid tracking:

- Generated files
- Downloaded dependencies
- Temporary files
- Passwords
- API keys



Undoing Mistakes

Git provides commands to safely fix mistakes.

git restore

Suppose you changed:

```
app.js
```

but you decide that you do not want those changes.

Run:



```
$ git restore app.js
```

This restores the file back to the last committed version.

Be careful

This command removes your uncommitted changes from the file.

Once those edits are discarded, they may not be recoverable.



Undoing Mistakes

Unstage a File

Suppose you ran:

```
● ● ●  
$ git add app.js
```

but you do not want to include it in the next commit.

Run:

```
● ● ●  
$ git restore --staged app.js
```

This removes the file from the staging area.

Importantly, your edits remain in the file.

So:

```
● ● ●  
$ git restore app.js
```



Undoing Mistakes

git revert

You can run:

Suppose you already committed something and later discover that the commit was a mistake.

```
● ● ●  
$ git revert 3f9e2d1
```

Git creates a new commit that reverses the previous commit.

The original commit still remains in the history.

This is useful when working with a team because it does not delete history.

git commit --amend

Suppose you made a commit but wrote a bad message.

For example:

```
stuff
```

You can change the most recent commit message:

```
● ● ●  
$ git commit --amend -m "Add user loading state"
```



Assignment 1: Build a Commit History

Students will use their Week 2 API mini-app.

The activity is worth 10 points.

Task 1: Configure Git

Use your real name:



```
$ git config --global user.name "Your Name"
```

Use your university email:



```
$ git config --global user.email "your@email.com"
```

Task 2: Create .gitignore

Create:

```
.gitignore
```



Assignment 1: Build a Commit History

It must contain at least:

```
node_modules/  
.env
```

Task 3: Initialize the Repository

Run:



```
$ git init
```

Then create your first commit.

For example:



```
$ git add .  
$ git commit -m "Initial commit"
```



Assignment 1: Build a Commit History

Task 4: Make at Least Three More Commits

Make changes to your project and commit each logical change separately.

For example:

```
Initial commit  
Add .gitignore  
Handle fetch errors  
Add loading state
```

Students should have at least four commits in total:

```
1 initial commit  
+  
3 additional commits
```

Task 5: Show Proof



Assignment 1: Build a Commit History

Run:



```
$ git log --oneline
```

Example:

```
8e1f0aa Add loading state  
5d4c3b2 Handle fetch errors  
2a1b0c9 Add .gitignore  
7f6e5d4 Initial commit
```

Then check the configured username:



```
$ git config user.name
```

Take screenshots showing this information.



Session Summary

After this session, students should understand the main Git workflow:





Session Summary

The most important commands from this session are:

```
git --version
git config
git init
git status
git add
git commit

git log --oneline
git diff
git diff --staged
git show
git restore
git revert
```

And the most important daily workflow is:



```
$ git status
$ git add .
$ git commit -m "Describe the change"
```

Students should also remember the role of:

```
.gitignore
```



Session Summary

which prevents unnecessary or sensitive files such as `node_modules/` and `.env` from being included in the repository.

NEXT SESSION The next session moves from local Git to collaboration: branching, merging, GitHub, pushing repositories online, and pull requests.